mb-free___faults-proceed

An AgentMPI run analysis

Abstract

This is the `PROCEED` point of the four-policy barrier-failure sweep in `bench_faults` (`experiments/microbench.py`): eight ranks, rank 3 returns immediately to simulate a crashed agent session, and the seven survivors enter a barrier with a 15 s deadline. `PROCEED` is the *partial barrier* — continue with whoever arrived, report the absentees, leave the communicator alone — and this run is the cleanest of the four by a wide margin. Thirty-nine events, 15.217 s, one repeated three-event sequence (`barrier.timeout`, `ft.declare_failed`, `coll.barrier`) executed independently by each of the seven survivors, and then the job ends. All seven agree on `absent: [3]` within 18.8 ms of one another, which is why the benchmark records `detected_correctly: true` and `all_completed: true` with a median detection latency of 15.195 s. Nothing unexpected happens, and that is the result: of the four policies, `PROCEED` is the only one that correctly names the dead rank without invoking any machinery that can then misbehave. Its sibling `SHRINK` detects identically and then spends a further 60.2 s in a broken agreement; `RAISE` and `WAIT` record no failure at all and blame the wrong rank. The single thing to take away is the division of labour that makes this work: `PROCEED` buys *knowledge* — a named absentee and a durable failure record — and deliberately defers the *repair* to the harness. The communicator ends the run at generation 0 with all eight members still marked `active`, so a second collective would wait for rank 3 all over again. That is a liability, but it is a bounded, honest and repeatable one, which is more than the other three policies can claim.

1 What this run is

property	value
run	`mb-free__faults-proceed`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	39
wall time	15.22 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	87.4%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	0	0 eager, 0 rendezvous
tokens sent	0	0 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

- [warning] `barrier/central` (label `phase`) was reached by 7 of 8 ranks, so it never completed.
- [warning] The coordination figures count time inside *completed* collectives only, and this run has ranks that never completed one. A rank records a collective on completion, so a rank that blocked and then timed out contributes nothing — the reported share is a floor, and on a badly degraded run a very loose one.
- [note] Collectives account for 87.4% of the run's rank-seconds, so more of the pool's time went to coordination than to work.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	0	0	0	0.0	finalized
1	0.00	0.0	0	0	0	0	0	0.0	finalized
2	0.00	0.0	0	0	0	0	0	0.0	finalized
3	0.00	0.0	0	0	0	0	0	0.0	finalized
4	0.00	0.0	0	0	0	0	0	0.0	finalized
5	0.00	0.0	0	0	0	0	0	0.0	finalized
6	0.00	0.0	0	0	0	0	0	0.0	finalized
7	0.00	0.0	0	0	0	0	0	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

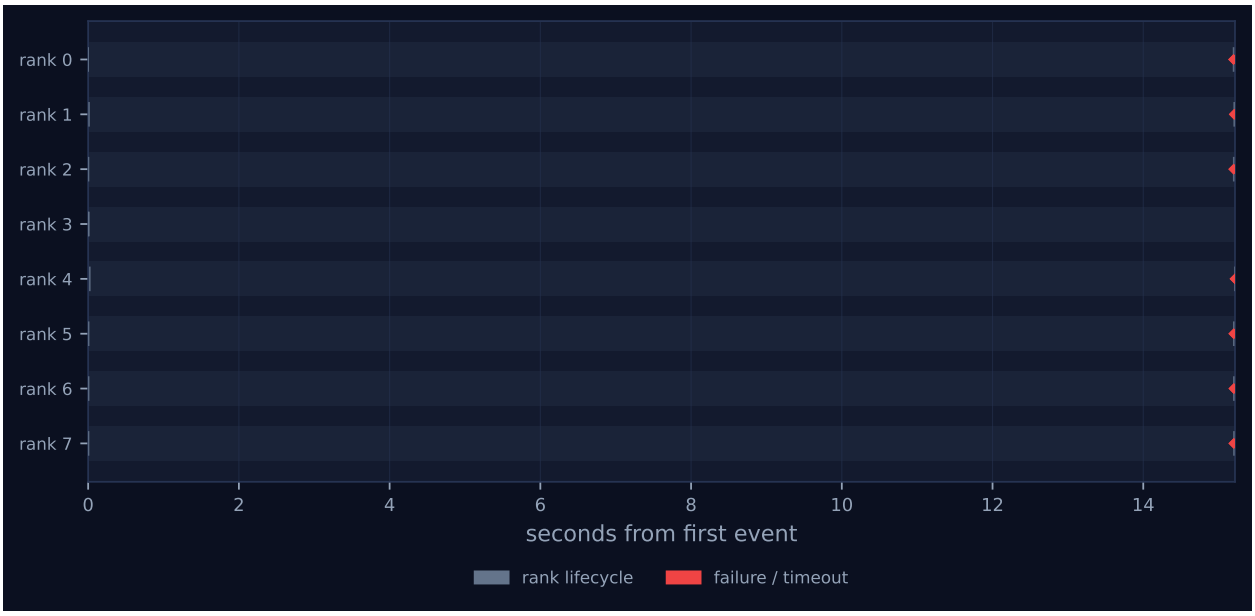

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
barrier	central	phase	8	7	0	2	0	0	0	15.20	0.02	–

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is a single vertical column of red at the right-hand edge and nothing else. Seven of the eight lanes carry one mark at $t \approx 15.2\,\text{s}$; rank 3's lane carries only its lifecycle ticks at $t \approx 0$. There are no bars anywhere, because there is no work anywhere: `metrics.json` records `executors: {none: 8}`, zero agent calls, zero tokens and $0.0000. The headline table's *achieved parallelism* 0.00× and *idle fraction 100.0%* are consequences of the benchmark's design and not findings about the protocol — `bench_faults`'s `rank_main` calls `comm.barrier` and nothing else, so a rank has nothing to be busy with. Read those two rows as "not applicable".

The same caution applies to the wall clock. 15.217 s is `-detect-timeout` (15.0 s, recorded in `results/microbench/free-faults.json`) plus 217 ms of poll granularity and SQLite write latency. The run measures a constant, and the residual measures the fabric.

Rank 3's empty lane is the injected fault in its entirety. It initialised at $t = 0.006\,\text{s}$ and finalised at the same millisecond, because `rank_main` returns `{"role": "victim"}` before touching the

communicator. From the survivors' point of view this is indistinguishable from an agent session that died before its first call, which is the point.

The interesting structure is inside the 18.8 ms width of that single column, and the viewer resolves it better than the figure does. The seven `barrier.timeout` events run from 15.198 s (rank 0) to 15.216 s (rank 4), and each rank then runs `ft.declare_failed`, `coll.barrier` and `rank.finalize` as one contiguous group in the log. Two things are visible in the millisecond timings. First, detection order is not completion order: rank 7 timed out at 15.199 s, third earliest, but did not get its declaration written until 15.203 s, behind ranks 2 and 5 — a 4 ms gap where every other rank's was under 1 ms. The declaration is an `INSERT` plus an `UPDATE` inside one `fabric.write()` transaction, so what that gap shows is the write lock queue. Second, the collective table records the arrival spread as *skew 0.02 s*, which is small absolutely but is 7× the 2.7 ms spread in the `SHRINK` sibling. The likeliest reading is entry skew propagated forward rather than detection jitter: a central barrier measures its deadline from each rank's own entry, and rank 4 both entered last (`rank.init` at $t = 0.022$ s, against 0.001 s for rank 0) and timed out last, 18 ms behind.

The viewer's stats row reads FAILURES 7, MESSAGES 0, and its legend counts *rank lifecycle 16, failure / timeout 14* with no third entry. The absence of the violet recovery colour is the visual signature of this policy: seven detections, zero recovery actions. Its COLLECTIVES panel lists `barrier`, 7 invocations, algorithm `central`, 0 messages — a collective that coordinated eight ranks without sending anything, because the central algorithm coordinates through `coll_parts` rows rather than through the message layer. Reproducing the barrier's 20 ms-to-250 ms poll back-off arithmetically gives about 70 polls per rank over 15 s, so roughly 490 `SELECT` statements crossed the fabric during the stretch of Figure 1 that looks entirely empty. That, and not any protocol cost, is what the wall clock's residual is made of.

The rank health table shows all eight ranks `finalized`, `alive: no`, `suspected: -`. The empty `suspected` column for rank 3 is worth noticing rather than glossing: it reflects `ft.health`'s lease-based detector, which this run never calls, not the `failures` table, which does hold rank 3.

7 Interpretation

7.1 What is true by construction

Almost the whole shape of this run was fixed before it started, and separating that from what emerged is most of the interpretive work.

The victim, the population size and the deadline are configuration: `victims: "3"`, `ranks: 8`, `detect_timeout: 15.0`. The algorithm is forced rather than chosen — `algorithms.barrier` overrides the caller's `dissemination` default with `central` whenever the policy is `PROCEED`, `SHRINK` or `REVOKE`, because only the arrival-counting algorithm can name who did not arrive. Everything downstream follows: no messages, no communication matrix figure, `eager_messages: 0`, and coordination conducted entirely through fabric rows. The barrier's `rounds: 0` against `predicted 2` in the collective table is a logging gap and not a disagreement: `algorithms.barrier` sets `rounds = 2` locally for the central path and returns it in `BarrierResult`, but never writes it onto the `_Tracker`'s `CollStats`, so the emitted `coll.barrier` always carries zero. The tooling declines to score it (–), which is right.

What emerged is narrower and worth stating precisely, because it is the run's actual content: seven independently polling processes, with no leader and no communication between them, converged on

the same absentee set within 18.8 ms and each recorded it. The consistency is not enforced anywhere. Each rank computes `absent = set(range(comm.size)) - arrived` from its own `SELECT`, and they agree because the fabric is a single serialising point of truth about arrivals. That is the design's central bet — route the hard part through a reliable shared service — and this run is a small, clean confirmation of it.

7.2 The mechanism: detection with a deliberately empty mitigation

`_central_barrier`'s deadline branch is common to `PROCEED`, `SHRINK` and `REVOKE`: emit `barrier.timeout`, then call `ft.declare_failed` for each absentee. The three policies then diverge, and `PROCEED` is the branch that does nothing further — it falls past the `REVOKE` and `SHRINK` tests and returns the absentee tuple. The barrier therefore returns *normally*, with a `BarrierResult` whose `absent` is `(3,)` and whose `complete` property is `False`, and the caller decides what that means. This is why the benchmark records `all_completed: true` and an empty `example_error` for this row: no rank raised.

The seven `ft.declare_failed` events reduce to one row in the fabric. The `INSERT` into `failures` carries `ON CONFLICT(ctx, rank, kind) DO NOTHING`, and querying `runs/mb-free/faults-proceed/fabric.sqlit` shows exactly one row: `rank 3`, `fail_stop`, `detail: "barrier phase"`. The `fabric.emit` call sits outside the conflict clause and fires unconditionally, so the event count is the number of *observers* and the row count is the number of *failures*. Seven observers, one failure. That is defensible for an event log, but it means the viewer's FAILURES tile of 7 and `metrics.json`'s `summary.failures: 7` both read as seven dead ranks when one rank died, and neither artefact says which it means. The `SHRINK` sibling makes the same arithmetic considerably worse, reaching 14 declarations for the same single failure.

7.3 What the policy costs: a repair that was never attempted

The fabric's end state is the most informative thing in this run, and it is invisible in every generated artefact. After the barrier, `comms.generation` is still `0`, `comms.revoked` is still `0`, and all eight `comm_members` rows — rank 3 included — are still `state = 'active'`. The communicator is bit-for-bit what it was before the failure, apart from one row in a side table.

That is exactly what the `BarrierPolicy.PROCEED` docstring promises: "continue with whatever contributions arrived, leaving the communicator intact and reporting the absentees." The liability is the direct consequence. `_central_barrier`'s completion test is `len(arrived) >= comm.size`, and `comm.size` is `len(self._members)`, which counts rank 3. A second barrier on this communicator would therefore wait the full timeout again and declare rank 3 failed again, and a message-passing collective — a `reduce`, say, whose default timeout is 1,800 s — would block on rank 3 for half an hour. A harness that calls a `PROCEED` barrier in a loop and never acts on the returned `absent` list pays 15 s per iteration forever.

I do not read this as a defect. It is a division of labour, and I think it is the right one: naming the absentee is something only the runtime can do, and deciding whether a seven-of-eight glossary is worth having is a domain judgement the protocol cannot make. The failure record is durable and machine-readable, so `ft.get_failed` will return `(3,)` to any later call and `ft.shrink`'s agreement step has the input it needs. What `PROCEED` guarantees is that the harness is *able* to decide; whether it does is not the barrier's problem.

It is worth contrasting this with what `SHRINK` does with the same information, because the comparison

is the sweep's main product. `SHRINK` takes the extra step of calling `ft.shrink_in_place`, and in that trace the extra step is where everything breaks: seven concurrent `UPDATE comms SET generation = generation + 1` statements drive the generation from 0 to 7, the seven survivors cache six different values, and the `agree` that follows splits across six separate collectives and never reaches quorum. `PROCEED` pays 15.2 s and gets a correct answer; `SHRINK` pays 75.4 s and gets a correct answer plus seven false failure declarations and an inconsistent agreement. The extra 60 s buys nothing.

7.4 The flagged findings

Three findings are generated and all three are expected for this configuration.

"*`barrier/central` (label `phase`) was reached by 7 of 8 ranks, so it never completed.*" This is the experiment, not a defect. `bench_faults` kills rank 3 on purpose precisely so that the barrier cannot complete; the policy under test is defined entirely by what happens at that moment, and a run of this benchmark in which the barrier *did* complete would be the broken one. The finding is also why the collective table shows – and `model_checks_total` is 0: `analysis.py` gates its cost-model comparison on `n_participants >= size` and correctly refuses to score a closed form against an invocation that did not happen.

"*The coordination figures count time inside* completed *collectives only… the reported share is a floor.*" Expected, and unusually tight here. The numerator behind *coordination share 87.4%* is 106.365 rank-seconds, which is 7×15.195 s — the seven survivors' barrier waits and nothing else. The only blocking the metric misses is rank 3's, which is zero, so on this run the floor is nearly the true value. That is a useful calibration point for the flag itself: it fires whenever a run has an incomplete collective, regardless of how much is actually being missed, and here it is formally correct but practically almost tight. In the `SHRINK` sibling the same flag hides four times the recorded blocking, and in `real-tr-p16-full` it hides a factor of about 4,500. The flag says "this is a floor"; it does not say how loose, and the three runs together show the range is enormous.

"*Collectives account for 87.4% of the run's rank-seconds, so more of the pool's time went to coordination than to work.*" Arithmetically correct and semantically empty. There was no work: zero agent calls, zero tokens. Any nonzero coordination figure would have exceeded the work figure. The 87.4% is worth citing only as a check that the run is what it claims to be — essentially all of it was one barrier, and *coordination in flight* of 99.98% says the same thing about wall clock. Read as a harness-efficiency finding it would be badly wrong, and it is the one flagged item here that a careless reader could misuse.

7.5 Against its siblings, and what a harness author should do

The four policies are the same program with one enum changed, so the differences between their traces are attributable to the policy and to nothing else. Set out that way the ordering is stark:

policy	barrier algorithm	events	wall (s)	`ft.declare_failed`	absentee correctly named
`wait`	dissemination	45	15.26	0	no
`raise`	dissemination	45	15.27	0	no
`proceed`	central	39	15.22	7	yes
`shrink`	central	60	75.43	14	yes, then seven false ones

WAIT and RAISE are indistinguishable from each other and diagnostically dead. Neither is in the list that switches the algorithm to central, so both run the dissemination barrier, which cannot name absentees. Their traces contain no barrier.timeout, no ft.* and no coll.* events; their fabrics' failures tables are empty; and both fail with ERR_TIMEOUT: receive timed out [...source=4 ...], naming a live rank rather than the dead one, because in a dissemination barrier at $p = 8$ the rank that notices is three rounds removed from the rank that died. The benchmark scores both detected_correctly: false.

The recommendation is PROCEED wherever a phase's input can legitimately be partial, and REVOKE plus an explicit out-of-place ft.shrink wherever it cannot — and not the SHRINK barrier policy, until the generation increment in shrink_in_place is fixed. This run is the direct evidence for the first half. It is the cheapest correct outcome in the sweep on every axis: fewest events, shortest wall clock, one failure row, no side effects the harness did not ask for, and an absent list handed back through a normal return. REVOKE is untested here, which is a genuine gap in the evidence, but it is the only escalating policy that does not route through shrink_in_place: it writes an absolute revoked = 1, which is idempotent under concurrent execution, and then raises, leaving the harness to call ft.shrink — the sibling that elects a leader and publishes one agreed context.

The archive supplies the counterexample that shows why this matters and, awkwardly, also shows that PROCEED alone is not sufficient. real-tr-p16-full is the only genuine failure among the 500 runs: sixteen ranks, all sixteen in failed_ranks, ok: false, 7,200 s of wall clock, $0.0726 spent and no output. Six rank roles were never bound to a worker process; the ten that worked correctly died waiting inside a glossary allreduce that could never fold. Its failures table has zero rows and its log has zero ft.declare_failed events — nothing was ever declared, so shrink's agreement step had no input and the harness had nothing to act on. Its own analysis concludes that a shrink at $t \approx 190$ s over the ten ranks that actually had executors would have finished the job in minutes. (Ten, not twelve: twelve is the number of ranks that emitted a coll.reduce record, which is a different set from the set that could do work.) The fifteen seconds in the present run are exactly the affordance that harness lacked.

But that harness *did* use PROCEED. It set the policy on a barrier in phase 5, downstream of the phase-2 allreduce it never survived to reach, and that placement is the whole difference. Reading the signatures in src/agentmpi/algorithms.py, policy is a parameter of barrier and of nothing else: bcast, scatter, gather, allgather, reduce, allreduce and reduce_scatter take a timeout and no policy at all. So no choice among these four names would have saved that run; the harness would have had to place a policy-bearing barrier *before* the allreduce, or the protocol would have to extend the policy to the other collectives. The lesson this sweep contributes is therefore not simply "choose PROCEED" but that a failure policy protects only the operation it is attached to, and that in AgentMPI today exactly one operation can carry one.

8 Threats to this reading

The injected fault is gentler than a crash in one specific, checkable way. Rank 3 returns normally, so it reaches rank.finalize at $t = 0.006$ s and its ranks row is finalized for the rest of the run. ft.declare_failed's state update is guarded by state NOT IN (FINALIZED, EXCLUDED), so when the survivors declared rank 3 dead at $t = 15.2$ s the guard skipped it: the fabric's ranks table ends the run with rank 3 finalized, not failed. A genuinely crashed agent would have been left running and the update would have fired. Nothing in this run's conclusions

depends on that state — the `failures` row was written either way, and it is the `failures` table that `get_failed`, `agree` and `_agree_survivors` consult — but a reader checking the rank table for evidence of the failure will not find it, and any harness that keys recovery off `RankState.FAILED` rather than off the failure records would see nothing here.

There are no agents in this run and nothing in it bears on agent behaviour. `executors: {none: 8}`, zero agent calls, zero tokens, $0. The failure modelled is a rank that returns instantly, which is `FAIL_STOP` and only `FAIL_STOP`. `src/agentmpi/ft.py`'s own taxonomy is explicit that the class that matters for agent systems is `FAIL_PLAUSIBLE` — a rank that returns a well-formed, confident, wrong answer — and that no amount of deadline tuning detects it. A `PROCEED` barrier would let such a rank through untouched, because it arrives. These four runs exercise one row of a six-row table, and a reader taking them as the evidence for the project's failure-model claims should scope them accordingly.

The detection latency is a restatement of the deadline. `detect_p50_s: 15.195` against a configured 15.0 s is the timeout plus 195 ms of poll granularity and fabric write latency. It is not a measurement of how quickly AgentMPI can notice a failure, because the only detector exercised is the barrier's own deadline. The lease-based detector that `ft.py` presents as *the* failure detector, `ft.health`, is never called in this run at all: the only caller is `_agree_survivors`, which `PROCEED` does not reach. Nothing here supports a detection-latency claim, and in particular nothing here says what would happen with a detect timeout short enough to race a slow-but-live rank.

The claim that a second collective would repeat the wait is read from the source, not observed. `rank_main` runs exactly one barrier, so this trace cannot show it. The inference is short — `comm_members` still reads all-active, `comm.size` counts all members, and `_central_barrier` compares against `comm.size` — but it is an inference, and a two-barrier variant of this benchmark would settle it in thirty seconds. The same experiment would settle the parallel question for `SHRINK`, where the situation is arguably worse: that policy *does* mark rank 3's member row `failed`, but `Communicator._active`, the tuple that records the distinction, is assigned in `refresh()` and never read anywhere in `src/agentmpi`.

The 18.8 ms skew attribution is plausible rather than proven. I read it as entry skew propagated forward, on the grounds that rank 4 was both the last to initialise ($t = 0.022$ s) and the last to time out ($t = 15.216$ s), and that a central barrier's deadline runs from each rank's own entry. The two orderings agree in six of seven positions — ranks 5 and 6 swap — which is consistent but is seven samples spanning 18.8 ms, and contention on the fabric's write lock is an equally good explanation for part of the spread, as rank 7's 4 ms detection-to-declaration gap shows directly. The distinction does not affect anything else in this reading.

$n = 1$, but the parts that matter are structural rather than statistical. A rerun would shuffle the millisecond ordering of the seven detections and move the skew. It would not change the event vocabulary, the one-row-seven-events asymmetry, the untouched communicator state, or the algorithm substitution, all of which are properties of the code rather than of this schedule. Where I have relied on a specific number — 18.8 ms, 15.195 s — I have tried to say what it is a measurement *of*, and in both cases the answer is the fabric and the configured deadline, not the protocol.